Family Meal Assistant

Functional Specification Document (FSD) — MVP

Document purpose
Define the functional, data, UX, recommendation, and non-functional requirements for the first releasable Android MVP of a family meal decision assistant.

Document Control

Document title	Family Meal Assistant — Functional Specification Document (MVP)
Version	1.0
Status	Final draft for product and engineering alignment
Platform	Android mobile application
Scope	MVP / Version 1
Prepared for	Product concept development and execution planning

At a Glance

Section	What it covers
1. Product definition	Purpose, problem, goals, users, scope, and guiding principles.
2. Functional design	Modules, user journeys, screen requirements, and acceptance criteria.
3. Recommendation logic	Candidate generation, scoring, learning signals, and image understanding.
4. Data and technical requirements	Core entities, APIs, analytics, privacy, and non-functional requirements.
5. Delivery scope	MVP boundaries, dependencies, rollout assumptions, and future expansion.

1. Product Definition

The product is an Android mobile application that helps a family decide what to cook next by remembering recently cooked meals, learning soft household preferences, and presenting practical suggestions for breakfast, lunch, dinner, tiffin, and snacks.

1.1 Problem Statement

In many households, one person repeatedly carries the cognitive load of planning the next meal. The difficulty is not only finding recipes; it is deciding what makes sense next for this family, at this time, without over-repeating the same dishes or choosing options that do not fit school, work, effort, or routine constraints.

1.2 Product Goal

Enable the primary cook or meal planner to answer a recurring daily question: what should I cook next for this family, at this time, without repeating too often?

1.3 Product Positioning

- Not a recipe-first app; it is a meal decision assistant.
- Not a heavy nutrition tracker; nutrition may be layered later.
- Not a grocery app in MVP; pantry and shopping are future extensions.
- Designed to learn quietly from meal history and small feedback signals.

1.4 Objectives

Objective	Why it matters	MVP success indicator
Reduce daily decision fatigue	The core user pain is repeated meal planning effort.	User views suggestions and acts on them during normal daily use.
Create useful family memory	History avoids accidental repetition and helps the user recall what worked.	Users log meals several times per week and consult recent history.
Deliver credible suggestions quickly	Recommendations must feel practical rather than generic.	Top suggestions receive taps, saves, or eventual meal logs.
Keep usage low-effort	If logging is heavy, retention will collapse.	First meal can be logged within the first session in under a minute.

1.5 Design Principles

The product should feel like a lightweight household helper, not a demanding data-entry system.

- Low effort: photo-first logging, minimal typing, and fast save.
- Quiet learning: infer patterns gradually instead of asking many questions.

- Practical over perfect: broad dish understanding is acceptable in MVP.
- Calm UI: simple navigation, clear next actions, and minimal clutter.
- Explainable suggestions: each recommendation should carry a human-readable reason.

1.6 Target Users

User type	Typical context	Primary needs
Primary cook / planner	Plans breakfast, tiffin, lunch, and dinner for the household.	Fast ideas, anti-repetition, and low-effort logging.
Parent managing school meals	Needs practical lunchbox ideas on school days.	Portable options, tiffin-friendly recall, and kid-accepted meals.
Working household decision-maker	Needs quick weekday meal choices after work.	Quick options and lightweight household memory.

1.7 Assumptions and Constraints

- The first release targets Android only.
- The MVP should function with minimal initial user history by relying on a starter meal catalog.
- The system may use cloud services for image understanding and recommendation logic, but the user experience should remain simple and responsive.
- Family preference learning in MVP is household-level, not deep per-person modeling.
- Recommendations must remain controlled and explainable; fully opaque black-box behavior is not required for MVP.

2. MVP Scope

2.1 In Scope

- Household onboarding and profile setup.
- Photo-first meal logging with meal type, timestamp, optional note, and optional meal name confirmation.
- Home screen with time-aware meal suggestions and recommendation reasons.
- Meal history with filtering by meal type and date grouping.
- Soft feedback signals such as Make again, Kids liked, Good for tiffin, Too much work, and Not a hit.
- Starter meal catalog for cold-start recommendation support.
- Basic rules-based recommendation engine with weighted scoring.

2.2 Explicitly Out of Scope

- Full recipe library or recipe authoring workflows.
- Pantry inventory, shopping list generation, or grocery ordering.
- Deep individual family-member profiles and nutrition plans.
- Voice input, social sharing, multilingual conversational assistant, or festival planning.

- Advanced collaborative filtering or sequence-heavy recommendation models.

2.3 MVP Definition of Done

The MVP is complete when a new user can onboard quickly, log meals with very little effort, view recent meal history, and receive 3–5 credible suggestions that become modestly more relevant through recency and feedback signals.

3. Functional Modules

3.1 Module Overview

Module	Primary user job	Core data	Priority
Onboarding & household setup	Set up the home context with minimum friction.	Household profile, diet type, meal types, school kids flag.	Must have
Home & recommendations	See what to cook next right now.	Current context, history, candidate meals, scores, reasons.	Must have
Add Meal	Capture what was cooked in seconds.	Photo, meal type, note, timestamp, suggested tags.	Must have
History	Recall what has already been cooked.	Meal entries, filters, quick feedback.	Must have
Soft feedback	Teach the system which meals worked.	Feedback tags, repetition signals, context fit.	Should have
Starter meal catalog	Provide reasonable options before much history exists.	Meal metadata and tags.	Must have

3.2 Onboarding and Household Setup

The onboarding flow must collect only the information needed to improve recommendations immediately. It should complete in roughly one to two minutes and avoid asking for deep family profiling.

Field	Required	Allowed values / format	Usage in product
Household/profile name	Yes	Free text, short	Display and account grouping
Number of family members	Yes	Integer	Planning context and future scaling
School-going kids	Yes	Yes / No	Tiffin relevance and weekday logic
Meal types to plan	Yes	Breakfast, lunch, dinner, tiffin, snack	Shows active planning contexts
Diet preference	Yes	Veg, egg, non-veg, mixed	Catalog filtering and suggestion eligibility
Cuisine preference	Optional	South Indian, North Indian, Mixed, Other	Biases starter catalog and future suggestions

Functional behavior

- The app shall allow setup with only required household-level fields.
- The app shall not force creation of individual profiles in MVP.
- The user shall be able to revise household settings later from Settings/Profile.
- The home screen shall be shown immediately after onboarding completes.

3.3 Home and Recommendations

The Home screen is the primary decision screen and must answer the question 'what next?' in a way that feels fast, trustworthy, and uncluttered.

Home screen content

- Current planning context switcher: breakfast, lunch, dinner, tiffin, snack.
- Top 3–5 meal suggestions for the selected context.
- Recommendation reason on every suggestion card.
- Recently cooked strip or list for quick anti-repetition scanning.
- Primary actions: Add meal, Cook this, Save for later, View history.

Functional behavior

- The app shall infer a default meal context from local time, but the user shall be able to switch context manually.
- The app shall show no fewer than three and no more than five suggestions by default.
- The app shall provide at least one short reason for each recommendation.
- The app shall avoid surfacing context-ineligible meals, such as non-portable items for tiffin when stronger alternatives exist.
- If the user has too little history, the app shall rely on the starter catalog filtered by household setup.

3.4 Add Meal

Meal logging must be optimized for speed. The preferred flow is photo first, then only the minimum confirmations required.

Input	Required	Default / source	Notes
Photo	Preferred but not mandatory	Camera or gallery	Used for broad recognition and easier recall in history
Meal type	Yes	User selection or system suggestion	Breakfast, lunch, dinner, tiffin, snack
Timestamp	Yes	Current date/time by default	Editable
Meal note	No	Free text	May include comments like kids loved it or easy tiffin
Meal name	No	System suggestion or manual entry	User can confirm, edit, or skip

Functional behavior

- The app shall save a meal even when image classification fails or is skipped.
- The app shall allow the user to confirm or override a suggested meal/category.
- The app shall support logging with partial information rather than blocking the save action.
- After save, the app may prompt a lightweight feedback action without making feedback mandatory.

3.5 Meal History

History serves both recall and anti-repetition. It must make it easy to answer 'haven't I already made that recently?'

Functional behavior

- The app shall display recent meals in reverse chronological order.
- The app shall group meals by date and clearly show meal type, meal name, and thumbnail/photo.
- The app shall support filtering by meal type.
- The app shall allow quick feedback actions directly from history cards or meal detail screens.

3.6 Soft Feedback Signals

Preference learning in MVP should stay subtle. Feedback should use simple, practical labels rather than detailed ratings or long forms.

Feedback signal	Meaning	Primary downstream impact
Make again	Strong positive household signal.	Boost future rankings.
Good for tiffin	Portable and practical for lunchbox use.	Boost tiffin-specific ranking.
Kids liked	Useful proxy for household acceptance.	Boost kid-relevant contexts.
Too much work	Meal is valid but effort-heavy.	Downrank in busy or weekday contexts.
Not a hit	Negative household outcome.	Downrank or exclude depending on repetition and recency.

3.7 Starter Meal Catalog

To solve cold start, the app needs a curated catalog of common meal ideas tagged with enough metadata to support filtering and ranking before the household has meaningful history.

Required metadata

- Meal name and normalized category
- Supported meal types
- Diet compatibility
- Tiffin suitability
- Effort level

- Light / heavy indicator
- Cuisine family or regional bias

4. Recommendation and AI Logic

The MVP recommendation system should be a hybrid of deterministic business rules and weighted scoring. The goal is not a large opaque AI model. The goal is explainable, context-aware ranking that improves quietly as usage data grows.

4.1 Recommendation Objectives

- Recommend meals that fit the current planning context.
- Avoid suggesting meals that were cooked too recently.
- Learn from household behavior without requiring heavy manual input.
- Explain why a suggestion is being shown.
- Remain controllable and tunable during MVP.

4.2 Input Signals

Signal group	Examples	Why it matters
Household context	Family size, diet type, school-going kids, cuisine preference	Defines the eligibility and context fit of candidate meals
Meal context	Breakfast/lunch/dinner/tiffin, weekday/weekend, time of day	Determines what types of meals are appropriate
History	Days since last cooked, frequency in last 7/30 days, prior meal slots	Drives anti-repetition and familiarity
Feedback	Make again, Good for tiffin, Too much work, Not a hit	Provides soft household preference signals
Catalog metadata	Effort, portability, light/heavy, cuisine, meal-type tags	Supports cold start and filtering
Implicit behavior	Suggested then chosen, suggested then ignored repeatedly, repeated soon after logging	Improves ranking quality without explicit effort

4.3 Recommendation Pipeline

Stage	Purpose	MVP approach	Output
Candidate generation	Assemble plausible meals for the current context.	Use starter catalog plus prior household meals.	Eligible meal set
Filtering	Remove ineligible items.	Apply diet, meal-type, and obvious context rules.	Filtered candidate set
Ranking	Sort candidates by likely usefulness.	Weighted scoring engine.	Ordered list of meals
Explanation	Translate score drivers into user-facing reasons.	Template-based reason generation.	Short recommendation reason

4.4 Candidate Generation Rules

- Include meals from the starter catalog that match diet type and meal context.
- Include normalized household meals that have been logged before and remain context-eligible.
- Exclude meals manually hidden or strongly downranked by repeated negative feedback, if such feature is added later.
- Prefer common and practical items over long-tail novelty in MVP.

4.5 Ranking Logic

Each eligible meal receives a weighted score. The top-ranked items become the visible recommendations. Suggested starting formula:

Recommendation score

- Score = context fit + recency gap bonus + positive preference bonus + tiffin bonus + effort suitability bonus + diversity bonus - repetition penalty - negative feedback penalty - context mismatch penalty

Factor	Direction	Example rule	Notes
Meal-time fit	Positive / gating	Breakfast items receive a strong boost for breakfast; dinner-only meals are not shown for tiffin.	Highest-priority driver
Recency gap	Positive	A meal not cooked for 10 days scores higher than one cooked yesterday.	Drives freshness
Repetition penalty	Negative	Same meal cooked in the same slot in the last 1–2 days should be heavily downranked.	High business value
Positive feedback	Positive	Make again and Kids liked both lift score.	Household-specific learning
Tiffin suitability	Positive	Portable items and Good for tiffin labels lift rank in tiffin context.	School/work utility
Effort suitability	Positive/Negative	Quick meals rank better for weekday mornings; effort-heavy meals lose points.	Context-sensitive practicality
Diversity balance	Positive	If recent meals were mostly rice-based, bread/roti-based options receive a modest boost.	Avoid monotony

Negative feedback	Negative	Not a hit and Too much work reduce score until contradicted by newer behavior.	Should not make items impossible forever

4.6 Suggested MVP Weighting Approach

Initial weights should be configurable from the backend or a ranking configuration file so the team can tune behavior without shipping a new app build.

Factor	Relative importance	Suggested starting range
Context fit	Very high	+25 to +35
Recency gap	High	+5 to +20
Positive feedback	Medium to high	+5 to +15
Tiffin suitability	Medium to high	+5 to +15
Effort suitability	Medium	+4 to +12
Diversity bonus	Low to medium	+2 to +8
Repetition penalty	Very high	−10 to −30
Negative feedback penalty	Medium to high	−8 to −20

4.7 Recommendation Reasons

The system should expose one short reason per visible suggestion by converting the top score drivers into natural language. Reasons should be simple, helpful, and non-technical.

- Not made in 9 days
- Worked well for tiffin before
- Quick option for busy mornings
- A good change from recent rice meals
- Family has liked this in the past

4.8 Cold Start Strategy

- Use the starter meal catalog as the primary source when household history is sparse.
- Bias starter suggestions using diet type, meal context, school kids flag, and cuisine preference.
- Begin weighting household behavior as soon as 5–10 logged meals exist.
- Do not wait for explicit ratings; repeated meal logging and suggestion selections should count as learning signals.

4.9 Image and Note Understanding

Image intelligence in MVP exists to reduce manual effort, not to achieve perfect dish identification.

Task	MVP approach	Expected output
Broad meal understanding	Food image classifier and/or multimodal service with shortlist	Likely meal name or category, confidence, and possible meal

	confirmation flow	type
Note normalization	Simple text parsing or language model normalization	Structured tags such as tiffin-friendly, kids liked, quick, light/heavy
Meal name normalization	Map variants like aloo sandwich and potato sandwich to a common representation	Cleaner history and recommendation learning

Image-understanding requirements

- The app shall support a suggestion-confirm-skip flow rather than requiring perfect classification.
- The system shall fall back gracefully if image confidence is low.
- The user shall always be able to save the meal without confirming the AI suggestion.

4.10 Learning Roadmap Beyond MVP

Phase	Model approach	Why
MVP	Rules plus weighted scoring	Explainable, data-efficient, easy to tune
Post-MVP personalization	Learning-to-rank model such as logistic regression or gradient boosted trees	Works well with tabular context and behavior features
Advanced personalization	Sequence-aware recommendation or embeddings	Useful only after meaningful scale and richer behavior data

5. User Flows

5.1 First-Time User Flow

1. User installs and opens the app.
2. User completes household setup with minimal required fields.
3. System lands the user on Home and selects a default meal context based on time.
4. System shows starter recommendations even if no meal history exists.
5. User may log a meal immediately or browse suggestions.

5.2 Add Meal Flow

6. User taps Add Meal from the bottom navigation or primary call-to-action.
7. User captures or selects a photo.
8. System suggests meal type and optionally a meal/category name.
9. User confirms or edits meal type, adds optional note, and saves.
10. System stores the meal and may present one-tap feedback tags.
11. History and recommendation signals update after save.

5.3 Recommendation Flow

12. User opens Home.
13. System sets or reads current planning context.
14. System generates candidate meals, ranks them, and displays the top 3–5.
15. User views reasons, opens a suggestion, saves it, or decides to cook it.
16. The interaction is recorded for analytics and optional future ranking signals.

5.4 History Flow

17. User opens History.
18. System displays recent meals grouped by date.
19. User filters by meal type if needed.
20. User opens a meal, adds soft feedback, or checks details before making the next decision.

6. Screen-Level Requirements

6.1 Navigation Model

Bottom navigation should be used in MVP with three primary tabs: Home, Add Meal, and History. Settings/Profile can be surfaced via a top-right entry point.

6.2 Home Screen

Area	Requirement	Notes
Meal context switcher	Must allow breakfast, lunch, dinner, tiffin, and snack selection.	Preselect based on time; user can override.
Suggestion list	Must show 3–5 cards with meal name, short reason, and primary	Keep visually calm and scannable.

	action.	
Recently cooked strip	Should show a compact recall view of recent meals.	Supports anti-repetition.
Primary CTA	Must provide Add Meal access from Home.	Prominent but not visually dominant.

6.3 Add Meal Screen

Area	Requirement	Notes
Camera / image picker	Must allow capture and gallery import.	Camera-first preferred.
Meal type selector	Must be quick to use and visible before save.	Chip or segmented control is preferred.
Image suggestion box	Should present likely meal/category without blocking save.	Allow edit or skip.
Optional note field	Should support short comments and context.	Keep low prominence.
Save action	Must be fast and always available if minimum required data is present.	Do not over-validate.

6.4 History Screen

Area	Requirement	Notes
Date grouping	Must group entries by date.	Today, Yesterday, or explicit date labels are acceptable.
Meal card	Must show thumbnail, meal name/category, meal type, and time/date.	Optimized for scanning.
Filters	Must support meal type filters.	Optional date range later.
Quick feedback	Should allow one-tap labels from list or detail view.	Improves learning with low effort.

6.5 Settings / Household Profile

- Edit household-level setup fields.
- View and manage diet type and meal contexts.
- Optionally manage future features such as reminders or favorites when introduced.
- Provide privacy, support, and data controls when those are implemented.

7. Detailed Functional Requirements

The following requirements are stated in implementation-ready terms. Priority values are relative to MVP delivery.

ID	Requirement	Priority	Acceptance criterion
FR-001	The system shall support creation of a single household profile during onboarding.	Must	A new user can complete setup and reach Home without creating individual member profiles.
FR-002	The system shall capture family size, school-going kids flag, meal types to plan, and diet preference during onboarding.	Must	These fields are stored and used in recommendation filtering.
FR-003	The system shall allow household settings to be edited after onboarding.	Should	Changes made in Settings are reflected in future recommendations.
FR-004	The system shall allow a user to log a meal with meal type and timestamp even if no photo is supplied.	Must	Meal entry can be saved with the minimum required fields.
FR-005	The system shall support camera capture and gallery import for meal images.	Must	User can choose either input path from Add Meal.
FR-006	The system shall suggest a broad meal name or category after image upload when confidence is adequate.	Should	A suggestion appears for supported images and can be edited or ignored.
FR-007	The system shall allow a user to save a meal even if AI meal understanding fails or is skipped.	Must	Save remains available; no blocking error is shown.
FR-008	The system shall record each meal with a normalized meal type value.	Must	Saved entries appear correctly in history and recommendation features.
FR-009	The system shall display meal history in reverse chronological order.	Must	Newest saved entries appear first on the History screen.
FR-010	The system shall support	Must	Selecting a filter updates

	filtering history by meal type.		the visible list correctly.
FR-011	The system shall generate recommendations for at least breakfast, lunch, dinner, tiffin, and snack contexts.	Must	Switching context updates the suggestion list.
FR-012	The system shall show between three and five ranked recommendations on Home.	Must	The Home screen never shows zero suggestions unless there is a system failure state.
FR-013	The system shall attach a short recommendation reason to each visible suggestion.	Must	Every suggestion card contains a readable reason string.
FR-014	The system shall apply diet filtering before ranking candidate meals.	Must	Meals incompatible with household diet are excluded.
FR-015	The system shall penalize meals that were cooked recently when ranking suggestions.	Must	A recently logged meal appears lower than an otherwise similar meal not cooked recently.
FR-016	The system shall boost meals previously marked Make again.	Should	A positive feedback signal changes future ranking behavior.
FR-017	The system shall boost meals or categories marked Good for tiffin in tiffin context.	Should	Tiffin-labeled meals rise relative to untagged items.
FR-018	The system shall downrank meals marked Too much work in busy contexts such as weekday mornings.	Should	Effort-heavy items appear below quick alternatives in weekday breakfast or tiffin scenarios.
FR-019	The system shall downrank meals marked Not a hit.	Should	Negatively tagged meals become less likely to appear soon after feedback.
FR-020	The system shall include a starter meal catalog to support recommendations before meaningful household history exists.	Must	A new household receives plausible suggestions immediately after onboarding.
FR-021	The system shall allow	Should	At least one supported

	users to add soft feedback from a saved meal or history entry.		feedback action can be saved after a meal entry exists.
FR-022	The system shall refresh ranking inputs after a meal is logged or feedback is added.	Must	Subsequent Home views reflect the updated data.
FR-023	The system shall persist meal history and household setup across app sessions.	Must	A returning user sees previously saved data after reopening the app.
FR-024	The system shall surface a friendly fallback state if recommendation generation fails.	Must	The user sees a recoverable message and still has access to History and Add Meal.

8. Data Model and API-Oriented Requirements

8.1 Core Entities

Entity	Key fields	Purpose
Household	household_id, family_size, has_school_kids, diet_type, active_meal_types, cuisine_preference	Stores household-level planning context
MealEntry	meal_id, household_id, meal_name, meal_category, meal_type, image_uri, note, logged_at, source	Stores what was cooked and when
Feedback	feedback_id, meal_id, feedback_type, created_at	Captures lightweight user feedback signals
CatalogMeal	catalog_id, meal_name, supported_meal_types, diet_type, tiffin_suitable, effort_level, light_heavy, cuisine_type	Supports cold start and ranking metadata
RecommendationEvent	event_id, household_id, meal_context, candidate_ids, shown_ids, selected_id, timestamp	Supports analytics and future model training

8.2 Derived Features for Ranking

- Days since last cooked
- Times cooked in last 7 days
- Times cooked in last 30 days
- Times cooked in the same meal slot recently

- Count of positive and negative feedback tags
- Whether a meal is known to be tiffin-suitable
- Effort suitability for the current context
- Recent category mix for diversity balancing

8.3 Suggested API Surface

Endpoint / service	Method	Purpose	MVP notes
/household	Create / update	Create or edit household profile	Can be local-first with sync later
/meals	Create / list	Save meals and retrieve history	Supports optional image upload path
/meals/classify	Post	Return meal/category suggestions from image and note	Non-blocking helper service
/feedback	Create	Store soft feedback against a meal	Lightweight write path
/recommendations	Get	Return ranked suggestions for a meal context	Central ranking service or local engine
/analytics/events	Post	Record exposure and interaction events	Can batch for efficiency

9. Non-Functional Requirements

Category	Requirement	Target / expectation	Priority
Performance	Home screen recommendation load	Under 2 seconds under normal network and data conditions	High
Performance	Meal save interaction	Under 2 seconds excluding heavy image upload latency	High
Reliability	Graceful failure	User can still add meals and view history if recommendations fail	High
Usability	Core flow simplicity	Primary tasks should be possible with one hand and minimal typing	High
Privacy	Default visibility	Family data and meal photos are private by default	High
Security	Basic transport/storage protection	Use standard authenticated APIs and	High

		secure image handling	
Scalability	Extensibility of ranking config	Weights and business rules should be externally configurable	Medium
Observability	Analytics and errors	Critical recommendation and save flows should emit monitoring events	Medium

9.1 Offline and Failure Considerations

- Meal logging should remain robust even when classification services are temporarily unavailable.
- The app should cache the household profile and recent history locally so previously saved content is visible on relaunch.
- Recommendation failure should fall back to a friendly empty or degraded state rather than a broken screen.

10. Analytics and Success Measurement

Metric area	Examples	Why it matters
Activation	Onboarding completion rate; first meal logged in first session	Measures setup friction and initial value
Engagement	Meals logged per week; recommendation views per active day	Measures day-to-day utility
Recommendation quality	Suggestion taps, save-for-later actions, meal logs after recommendation exposure	Validates ranking usefulness
Retention	Week-2 and week-4 retention	Shows whether the product becomes habit-forming
Learning coverage	Share of meals with at least one feedback or implicit learning signal	Measures whether quiet learning is working

11. Dependencies, Risks, and Mitigations

Risk / dependency	Impact	Mitigation
Low logging frequency	Weak household history and poor personalization	Keep Add Meal extremely fast; never block save with excessive fields
Weak image recognition	Frustrating or incorrect meal suggestions during logging	Use suggestion-confirm-skip flow and broad categories, not exact forced recognition

Generic recommendations	Low trust and weak retention	Use recency, context fit, and recommendation reasons aggressively from day one
Over-complex onboarding	Early drop-off	Restrict MVP setup to a few required household fields
Sparse data in cold start	Poor first impression	Use curated starter catalog filtered by household context

12. MVP Acceptance Summary

The MVP should be accepted for release readiness when the following are true:

- A new household can onboard and reach Home with starter recommendations in the first session.
- A user can log a meal with very few steps and without mandatory AI confirmation.
- History makes recent meals easy to review by date and meal type.
- The app returns 3–5 context-appropriate recommendations with short reasons.
- Ranking changes in sensible ways when meals are logged or feedback is added.
- Failure states are graceful and do not block core app usage.

13. Future Extensions (Not in MVP)

- Ingredient-aware suggestions and pantry context
- Leftover reuse recommendations
- Weekly meal memory calendar and meal planning
- Deeper family-member preference modeling
- Nutrition overlays and health goals
- Learning-to-rank personalization model
- Regional meal packs and multilingual support

Appendix A. Example Recommendation Scenario

Scenario: Tuesday evening; household is vegetarian; two school-going kids; tiffin is being planned for tomorrow morning. Recent tiffins this week were paratha, sandwich, and lemon rice.

Candidate	Key signals	Ranking effect	Expected result
Paneer roll	Good tiffin fit; not made recently; portable; likely kid-friendly	Strong positive	Top suggestion
Veg sandwich	Good fit but used recently	Moderate positive with recency penalty	Middle suggestion
Lemon rice	Good tiffin fit but made	Penalty for repetition	Downranked

	recently		
Poha	Better breakfast than tiffin	Context mismatch	Low rank
Idli podi lunchbox	Portable and distinct from recent patterns	Diversity benefit	Strong alternate suggestion